

Hiring Assignment — Software Developer (Intern + Full-Time)

Backend pre-assignment · Role-based access control for a reimbursements tool

About us

Razorpay is a FinTech company. Our products help small businesses and firms around the world keep their finances in order — tracking money in and out, and making the day-to-day of running a company a little less painful. We work with real money and real people, so correctness, clarity, and trust matter to us more than anything flashy.

What we are looking for

We are hiring developers who like to own their work end to end. Beyond writing code that runs, we care about how you think. Specifically, we look for:

- **Problem solving** — you can break a fuzzy problem down and reason your way to a sound solution.
- **Ownership** — you treat the task as yours, follow it through, and sweat the details no one explicitly asked about.
- **Strong computer-science fundamentals** — you understand what is happening under the hood, not just which library to call.
- **Fire-fighting ability** — when something breaks in production, you can stay calm, debug methodically, and fix it.
- **Solid OOP and Low-Level Design** — you model your domain well and write code that another engineer can extend without fear.

The assignment & ground rules

Please read this section carefully before you start. A few of these points are about how we evaluate you; others are hard requirements that our testing team depends on.

- **AI tools are allowed.** Use whatever helps you work — but remember that you are responsible for everything you submit, and we will ask about your choices.
- **Submit as a GitHub repository.** The link to the submission Google Form has already been sent to the email address of shortlisted candidates.
- **Code quality and maintainability are judged.** Choose the right architecture for the problem and follow SOLID principles. We would rather read clean, well-structured code than clever code.
- **Tech stack.** Our work mostly revolves around JavaScript and Golang. For this assignment, build it in **JavaScript (no TypeScript)**. The allowed frameworks are **Express.js** or **Nest.js**.
- **This is a pre-assignment and is backend only.** Candidates who score above our threshold will be shortlisted for a final assignment — a full-stack task built as an extension of this backend, so structure your work with that in mind.
- **Database.** Use any SQL database. **PostgreSQL is preferred.**

TESTER REQUIREMENTS — READ THIS TWICE

Our testing team will run your application and execute an automated API test suite against it. If your project does not match the conventions below, the tests will not be able to talk to your app — and your score will suffer for it.

Your repository root must contain a `package.json` exposing these scripts:

- `dev` — starts the backend app. It **must** run strictly on **port 7002**.
- `db:migrate` — sets up the required database structure on the tester's system.
- `db:seed-data` — sets up the required CFO account. **Do not seed anything other than the CFO account** (the root user).

It is your responsibility to have the migration files ready and runnable. The expected repository shape is:

```
your-repo/  
  package.json      # exposes: dev, db:migrate, db:seed-data  
  ...migration files # ready and runnable
```

- Node version must be **>= 20.10.2**.
- Because the tester runs an API test suite, **deviating from the conventions in this document — endpoints, routes, request and response shapes — can drop your score drastically**. Stick to them exactly.

The task

Build the backend for a **reimbursements management tool** for one of our partner organizations. The whole point of the exercise is role-based access control: who can see what, and who can do what.

Stakeholders & roles

Stakeholder	Role string	Notes
Employee	EMP	Default role for every new account.
Reporting Manager	RM	A group of employees reports to one RM.
Accounts Payable Executive	APE	Handles payable-side approvals.
Chief Financial Officer	CFO	Root user. Seeded, not registered.

CFO (ROOT) CREDENTIALS — SEED EXACTLY THIS

email cfo@org.com
password CFO#ORG@April2026

Organization structure

- A group of employees reports to exactly one RM.
- Every employee reports to exactly one RM.
- There is **no direct relationship** between:
 - EMP and APE
 - RM and APE
 - APE and CFO

API specification

All endpoints are under the `/rest` prefix. Authentication is cookie-based: the login response sets the auth cookie, and protected endpoints read it automatically. Follow these routes, request bodies, and response shapes precisely.

PUBLIC ENDPOINTS

POST `/rest/onboardings/register`

Self-service registration. Every account created this way starts with the default `EMP` role.

REQUEST BODY (JSON)

Employee's `name`, `email`, and `password`.

RULES

Only allow emails on the `org.com` domain. Assume the email is valid and belongs to the user — any SMTP / verification work is out of scope for this task.

POST `/rest/onboardings/login`

REQUEST BODY (JSON)

Employee's `email` and `password`.

RULES

Only allow emails on the `org.com` domain.

POST `/rest/onboardings/logout`

No explicit payload — relies on the auto-tagged cookies sent with the request.

PROTECTED ENDPOINTS

POST `/rest/roles/assign`

Assigns a role to a user. We currently have a fixed set of roles: `EMP`, `RM`, `APE`, `CFO`.

WHO CAN CALL THIS

Only the **CFO**.

REQUEST BODY (JSON)

`role` (the exact role string).

GET /rest/employees

VISIBILITY BY ROLE

- **EMP** — not allowed to access this API.
- **RM** — lists all the EMPs reporting to them.
- **APE** — lists all EMPs and RMs.
- **CFO** — lists everyone.

SUCCESS RESPONSE

```
{
  "status": "success",
  "data": {
    "users": [
      {
        "userId": ...,
        "name": ...,
        "email": ...,
        "role": ...
      },
      ...
    ]
  }
}
```

POST /rest/employees/assign

Assigns an employee to a reporting manager.

WHO CAN CALL THIS

Only the **CFO**.

REQUEST BODY (JSON)

`userId` (of the EMP), `userId` (of the RM).

DELETE /rest/employees/assign

Removes the assignment between an employee and a reporting manager.

WHO CAN CALL THIS

Only the **CFO**.

REQUEST BODY (JSON)

`userId` (of the EMP), `userId` (of the RM).

REIMBURSEMENT STATUS

A reimbursement's status is one of: `PENDING`, `REJECTED`, `APPROVED`.

`POST` `/rest/reimbursements`

Raises a new reimbursement.

WHO CAN CALL THIS

Only an **EMP** can create a new reimbursement.

REQUEST BODY (JSON)

`title`, `description`, `amount`.

BEHAVIOR

A newly raised reimbursement has status `PENDING`.

`PATCH` `/rest/reimbursements`

RULES BY ROLE

- **EMP** cannot edit a reimbursement once it has been raised.
- **RM** can approve or reject the reimbursement.
- **APE** can approve or reject the reimbursement.
- **CFO** can approve or reject the reimbursement.

REQUEST BODY (JSON)

`userId`, `status`.

APPROVAL RULE

In the EMP's view, the status turns `APPROVED` only once their **RM** **and** one of the APEs have approved it.

GET /rest/reimbursements

VISIBILITY BY ROLE

- **EMP** — lists their own reimbursements.
- **RM** — sees the reimbursements that are **PENDING** from their EMPs.
- **APE** — sees reimbursements **PENDING** at the APE level but already **APPROVED** by the RM.
- **CFO** — sees reimbursements already **APPROVED** by the APEs.

SUCCESS RESPONSE

```
{
  "status": "success",
  "data": {
    "reimbursements": [
      {
        "title": "abcd",
        "description": "xyz",
        "amount": 999,
        "status": "..."
      },
      ...
    ]
  }
}
```

GET /rest/reimbursements/<user-id>

Lists all the reimbursements for the given **user-id** — but only if that user is an EMP and is the requesting user's subordinate.

SUCCESS RESPONSE

```
{
  "status": "success",
  "data": {
    "reimbursements": [
      {
        "title": "abcd",
        "description": "xyz",
        "amount": 999,
        "status": "..."
      },
      ...
    ]
  }
}
```

Code checking guidelines

Since AI tools are allowed, we pay close attention to *how* you built the project — not just the final result. A few things we will be looking at:

1. We will analyze your code the way it was built, so write it the way you would actually ship it.
2. We expect a clean commit history that lets us follow how you went from setting up the project to implementing each feature.
3. A high number of commits is not what we are after — **clean, structured, incremental project building** is.
4. Our tester will run your backend against an API test suite created specifically for this assignment.
5. You will be shortlisted for the next round only if we can see that you used your **own problem-solving to frame the solution and leveraged AI to write the code**.
6. Maintain an incremental `prompt-logs.txt` file in the project root. Append every prompt you use to it as you go — it should grow incrementally and be committed along with each commit.

How you'll be evaluated

There is no single "correct" submission. We are reading your code to understand how you build software. In particular:

- **Code quality and structure** — clear naming, sensible file organization, and code another engineer could pick up and extend.
- **Architecture and design** — the right separation of concerns and faithful adherence to SOLID principles.
- **Correctness** — the endpoints behave exactly as specified, with the access rules enforced per role.
- **Edge-case coverage** — the more thoughtfully you handle the awkward and unhappy paths, the higher you score. This is where strong candidates separate themselves.

All the best — we are looking forward to reading your work.